Olio-relaatiokuvaus

Lähttökohdat. Spring Boot. Spring Web.
Spring Data JPA.

Oppimistavoitteet

1. Ymmärretään ORM-kerroksen toteutuksen lähtökohdat ja tavoitteet
2. Opitaan ORM-kerrosta tukevien kehysten käytön perusteet

Lähtökohdat

- Aiemmissa opinnoissa on opittu ORM-perusteet JPA-määrityksen mukaisesti.
  - Toteuttavana teknologiana Hibernate
- Tällä opintojaksolla syvennetään JPA-osaamista sekä opitaan ORM-kerrosta käyttävän sovelluksen toteutusta helpottavia teknologioita. Erityisesti:
  - Spring Boot
  - Spring Web
  - Spring Data JPA



Spring Boot

- Mikä on Spring Boot?
  - Java-pohjainen kehitysalusta, joka helpottaa ja nopeuttaa web-sovellusten ja taustajärjestelmien (backendien) rakentamista.
  - Rakentuu suositun Spring Frameworkin päälle, mutta vähentää konfiguroinnin ja käynnistyksen vaivaa (“auto-configuration”).
  - Spring Bootilla saat valmiin rakenteen, automaattikääntämisen sekä omat ”sisäänrakennetut” työkalut mm. web-rajapinnoille, tietokannalle ja testaukselle.
- Spring Bootin peruspiirteet:
  - Yksi käynnistettävä paketti (JAR), jossa kaikki tarvittava mukana (sisään leivottu sovelluspalvelin).
  - Vähemmän konfiguroitavaa: *Convention over configuration*.
  - Yhteensopiva Springin lisäosille, kuten Spring Web (REST-rajapinnat) ja Spring Data JPA (tietokantaintegraatio).
  - Sovellus voidaan helposti käynnistää, testata ja ottaa käyttöön yhdellä komennolla.
- Mihin Spring Bootia käytetään?
  - Nopeat verkkopalvelut ja taustajärjestelmät ("API-backendit")
  - Datakeskeiset sovellukset, joissa tarvitaan integraatio tietokantaan
  - Lähestymistavaltaan moderni, helposti ylläpidettävä ohjelmistokehys

Spring Web

- Mikä on Spring Web?
  - Spring Web on Spring Bootin osa, jonka avulla rakennetaan HTTP-rajapintoja (eli REST API -palveluita) sekä verkkosovelluksia.
  - Spring Web -moduuli tarjoaa selkeän tavan määritellä, miten sovellus vastaa erilaisiin verkkopyyntöihin (GET, POST, PUT, DELETE...).
- Keskeiset käsitteet:
  - Controller: Sovelluksen “ovet” – vastaanottaa HTTP-pyynnöt ja palauttaa vastaukset.
  - @RestController: Merkintä luokalle, joka palvelee REST-rajapintana (palauttaa tiedot suoraan, tyypillisesti JSON-muodossa).
  - @RequestMapping, @GetMapping, @PostMapping...: Kuvataan tarkkaan, mihin polkuihin ja HTTP-metodeihin sovellus vastaa.
  - DTO (Data Transfer Object): Rakenne, jolla tiedot siirtyvät palvelusta asiakkaalle ja päinvastoin.
- Spring Web REST API:n “elinkaari”:
  - Selaimen/käyttäjän ohjelman tekemä HTTP-pyyntö (esim. GET /api/products/123)
  - Controller-luokka vastaanottaa pyynnön (esim. ProductController)
  - Sovellus käsittelee datan (esim. hakee tuotetiedot tietokannasta)
  - Palauttaa vastauksen HTTP-muodossa (esim. JSON-muotoinen tuote-objekti)
- Mihin Spring Webiä käytetään?
  - Verkkopalvelinten ja taustajärjestelmien ohjelmointiin: API-toteutukset, tiedon esittäminen ja muokkaus webin kautta
  - Sovellusten liittämiseen toisiinsa HTTP-rajapintojen avulla
  - Selaimen, mobiilisovelluksen tms. viestintään taustapalvelimen kanssa

Spring Data JPA

- Mikä on Spring Data JPA?
  - Spring Data JPA on Spring Bootin osa, joka helpottaa tietojen tallentamista ja hakemista relaatiotietokannasta suoraan Java-olioina (ORM, Object-Relational Mapping).
  - Taustalla toimii JPA (Java Persistence API) ja tyypillisesti Hibernate, jotka hoitavat automaattisesti olioiden ja tietokannan taulujen välisten yhteyksien synkronoinnin.
- Keskeiset käsitteet:
  - Entity: Java-luokka, joka vastaa tietokannan taulua (@Entity-annotaatio).
  - Repository: Rajapinta, jonka avulla CRUD-toiminnot hoituvat ilman SQL:n kirjoittamista (@Repository, JpaRepository).
  - @Id, @Column, @OneToMany…: Annotaatiot, joilla määritellään olion kenttien ja taulujen/suhteiden vastaavuus.
  - JPQL: Kyselykieli, jota käytetään olioiden hakemiseen (Java Persistence Query Language).
- Spring Data JPA:n “elinkaari”:
  - Sovellus kutsuu repositoryn metodia (esim. findById(123)).
  - Spring Data JPA generoi tarvittavan SQL:n automaattisesti ja suorittaa sen tietokannassa.
  - Tulos muunnetaan automaattisesti oikeaksi Java-olioksi, joka palautetaan sovellukseen.
  - Muutokset (esim. save, delete) päivitetään tietokantaan ilman manuaalista SQL:ää.
- Mihin Spring Data JPA:ta käytetään?
  - Sovelluksen olioiden helpoon tallentamiseen/hakuun tauluista ilman SQL-koodin kirjoittamista.
  - Liiketoimintalogiikan rakentamiseen, jossa tietokantatoiminnot pysyvät selkeinä ja erillään muusta koodista.
  - Yhdistää Spring Boot -sovelluksen suoraan tehokkaasti relaatiotietokantaan.

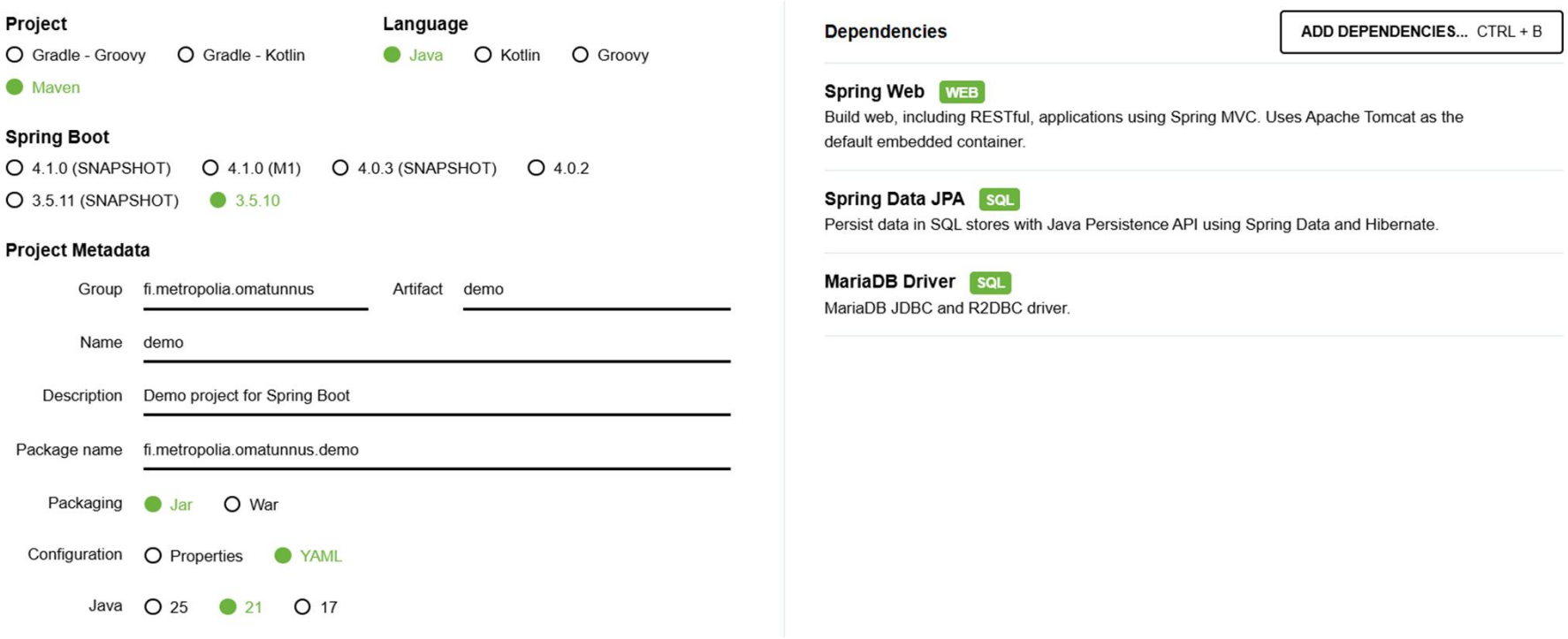
Demo

- Luodaan pankki-tietokantaa käyttävä sovellus, jolla voidaan hakea tilien saldoja.
- Valmistelu: tee tietokantaan käyttäjätunnus sovellusta varten ja määrittele sille oikeudet
  - CREATE USER
  - GRANT

pankki.tili: 3 riviä yhteensä (exact)

#	id	saldo
1	1	100,0
2	2	5 000,0
3	3	0,25

Projektin luonti



- Projekti luodaan osoitteessa https://start.spring.io.
  - Tämä on suositeltava tapa.
  - Tee asetukset kuvan mukaisesti (group, artifact ja name kuitenkin yksilöllisesti).
- Pura zip ja sen jälkeen avaa projekti kehittimessä: File / Open

Demo: tekniikan kokeilu

3. Toimiiko Hello World?
   - Lisätään HelloController.java samaan pakkaukseen, jossa sovellus sijaitsee.
   - Määritetään yksi endpoint: /
   - Aja sovellus Run-toiminnolla ja avaa selaimessa **`localhost:8080/`**

```java
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    @GetMapping("/")
    public String hello() {
        return "Hello, World!";
    }
}
```

Demo: annotoitu kohdeluokka

1. Tee entity-pakkaus lähdekoodipakkauksen (esim. fi.metropolia.omatunnus) sisälle
2. Lisää entity-pakkaukseen annotoitu Tili-luokka

```
package com.example.demo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "person")
public class Person {
   @Id
   private Integer id;

   private String firstname;
   private String lastname;

   // lisää oletuskonstruktori, getterit ja setterit
}
```

Demo: repositorio

```
package com.example.demo.entity;

import org.springframework.data.jpa.repository.JpaRepository;

public interface TiliRepository extends JpaRepository<Tili, Integer>
{
}
```

1. Tee repositorio tilille rajapintana, joka periytetään valmiista JpaRepository-rajapinnasta
   - Tyyppiparametreina annetaan kohdeluokan tyyppi (Tili) ja siellä määritetyn tunnistavan ominaisuuden, eli tilin id:n, tyyppi (Integer)
   - Tämän ansiosta saadaan automaattisesti keskeiset datan käsittelymetodit. Olio, jolla nuo metodit on toteutettu, luodaan ajonaikaisesti ja automaattisesti.

Demo: kontrolleri

- Kontrolleri tuottaa JSON-esityksen oliosta.
- Jos haluat Tili-olion, voit kirjoittaa esim.

```
Tili tili =
repository.findById(id)
.orElse(null);
```

```
package fi.metropolia.olliv.tkrdemo;

import fi.metropolia.olliv.tkrdemo.entity.Tili;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/tili")
public class TiliController {

  private final TiliRepository repository;

  public TiliController(TiliRepository repository) {
    this.repository = repository;
  }

  @GetMapping("/{id}")
  public ResponseEntity<Tili> getTiliById(@PathVariable Integer id) {
    return repository.findById(id)
        .map(tili -> ResponseEntity.ok(tili))
        .orElse(ResponseEntity.notFound().build());
  }
}
```

Metropolia

Demo: Tietokantamääritys

- Tehdään määritys resources-kansion YAML-tiedostoon application.yml (ks. kuva)

```yaml
spring:
  jpa:
    properties:
      hibernate:
        show_sql: true
        format_sql: true
        ddl-auto: validate

  application:
    name: demo
  datasource:
    url: jdbc:mariadb://localhost:3306/pankki
    username: dbuser
    password: nuifghidsufhfERGWEG323
```

Demo: kokeilu

- Testataan lopuksi tietokantahaun toimivuus.
- Nyt koneiston päälle voi rakentaa kokonaisen tietokantasovelluksen.

